# On-Device Characterization and Latency Profiling of Ultra-Low-Resource INT8 TinyML Models on ESP32 Microcontrollers

Narendra Satish

Abstract—Deploying deep learning on resource-constrained 32-bit microcontrollers (TinyML) requires empirical hardware characterization to bridge the translation gap between host simulation and physical silicon. However, tabular diagnostic models ($<$ 4 KB Flash, sub-100 µs latency) remain under-characterized on commercial silicon. This paper presents an on-device empirical characterization of four disk-verified FULL_INT8 MLP models deployed on an Espressif ESP32-D0WD-V3 microcontroller (Xtensa LX6 @ 240 MHz, 4 MB Flash, 320 KB SRAM). Using a zero-I/O in-RAM hardware timer benchmarking protocol across 24,000 measured single-sample inferences, we show: (1) isolated on-device latency scales monotonically with parameter count ($R^2 = 0.963$) from 64.55 µs (176 parameters) to 89.90 µs (412 parameters); (2) structural distillation delivers an observed 28.20% latency reduction relative to the baseline; (3) host x86_64 profiling underestimates microcontroller latency by 62.9×–76.8× and introduces sub-microsecond rank inversions unmasked on silicon; and (4) TensorFlow Lite Micro commits exactly 916 Bytes of tensor arena memory with zero dynamic heap allocations across 25,200 executions. These results provide defensible empirical baselines for edge intelligence under real-time cyber-physical constraints.

Index Terms—TinyML, Microcontroller Benchmarking, ESP32, Xtensa LX6, Integer Quantization, Latency Profiling, Embedded ML.

## I. Introduction

THE rapid deployment of Tiny Machine Learning (TinyML) on low-power microcontrollers (MCUs) enables real-time intelligence directly at the extreme edge [1]–[3]. In high-frequency sensor applications such as automotive powertrain diagnostics and vibration monitoring, executing neural network inference on-device eliminates latency penalties, communication energy, and cloud dependency [4], [5].

However, evaluating TinyML models strictly within host-side training frameworks introduces a significant deployment reality gap [6], [7]. Host x86_64 CPUs utilize deep out-of-order superscalar pipelines and multi-megabyte cache hierarchies that do not reflect embedded execution. Furthermore, existing TinyML benchmarks predominantly focus on vision and keyword-spotting models ($>$ 100 KB Flash, 10–300 ms latency) [4], [8], [9], leaving ultra-low-resource tabular diagnostic networks ($<$ 4 KB Flash, sub-100 µs execution) insufficiently characterized.

Manuscript received August 29, 2026. (Corresponding author: Narendra Satish, e-mail: narendresh.p@gmail.com). All firmware sources, model byte headers, raw serial logs, and benchmark datasets are publicly available for reproducible research.

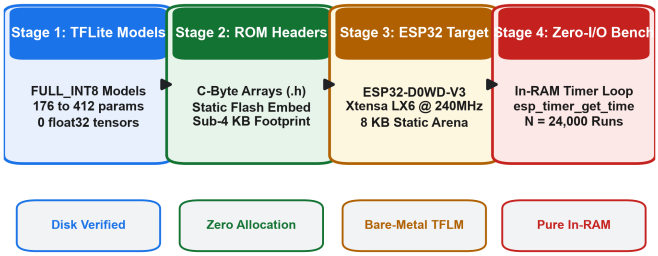


Fig. 1. The physical deployment and zero-I/O in-RAM benchmarking pipeline connecting verified FULL_INT8 TFLite FlatBuffers to bare-metal ESP32 silicon.

In this paper, we bridge this translation gap by providing an empirical on-device characterization and latency profiling of four verified, disk-serialized FULL_INT8 TinyML models deployed to an Espressif ESP32-D0WD-V3 microcontroller (Xtensa LX6 @ 240 MHz). Figure 1 illustrates the physical deployment and measurement workflow. We formulate four concrete Research Questions: RQ1 (On-Device Latency): How do evaluated FULL_INT8 models differ in physical inference latency on ESP32-D0WD-V3? RQ2 (Compute Translation): How do parameters and MAC counts relate to on-device latency? RQ3 (Host Divergence): How does host x86_64 latency differ from microcontroller execution? RQ4 (Memory & Determinism): What Flash, static SRAM, tensor arena, and heap footprints are observed during inference?

## II. Hardware and Deployment Architecture

### A. Physical Silicon Platform

All physical benchmarks were executed on an Espressif ESP32-D0WD-V3 microcontroller (silicon revision v3.1). Table II summarizes the audited hardware and software environment. The chip features dual 32-bit Xtensa LX6 microprocessor cores operating at a fixed 240 MHz clock frequency (4.167 ns cycle period) with a 40 MHz crystal reference. The board integrates 4 MB of external Quad-SPI Flash (80 MHz QIO mode) and 320 KB of internal on-chip SRAM. External Pseudo-Static RAM (PSRAM) was not present, ensuring all tensor buffers and runtime stacks executed purely within internal on-chip SRAM.

### B. Evaluated Model Portfolio

We evaluated four multi-layer perceptron models trained on multi-sensor internal combustion engine diag-

TABLE I
Empirical On-Device Single-Sample Inference Latency Statistics on ESP32-D0WD-V3 (240 MHz, $N = 24,000$ Measured Trials)

| Model Identifier | Params | Mean Latency | Median (P50) | Std. Dev. | P95 Latency | P99 Latency | Min | Max | IQR | CV (%) |
|---|---|---|---|---|---|---|---|---|---|---|
| student_a_8_4_int8 | 176 | 64.55 µs | 64.00 µs | 3.73 µs | 69.00 µs | 76.00 µs | 64 µs | 77 µs | 2.0 µs | 5.78% |
| student_b_16_4_int8 | 328 | 72.96 µs | 72.00 µs | 4.96 µs | 83.00 µs | 83.00 µs | 72 µs | 84 µs | 2.0 µs | 6.80% |
| mlp_12f_int8 | 380 | 76.77 µs | 77.00 µs | 3.65 µs | 83.00 µs | 90.00 µs | 76 µs | 90 µs | 2.0 µs | 4.75% |
| mlp_14f_int8 | 412 | 89.90 µs | 90.00 µs | 2.66 µs | 95.00 µs | 101.00 µs | 88 µs | 102 µs | 2.0 µs | 2.96% |

TABLE II
Audited Hardware and Software Deployment Configuration

| Property | Specification | Property | Specification |
|---|---|---|---|
| MCU Silicon | ESP32-D0WD-V3 (v3.1) | Flash ROM | 4 MB Quad-SPI (80 MHz) |
| CPU Core | Dual Xtensa LX6 @ 240 MHz | SRAM | 320 KB Static (0 PSRAM) |
| Crystal | 40 MHz Oscillator | UART Bridge | WCH CH9102 (COM7, 115.2k) |
| Framework | Arduino ESP32 / FreeRTOS | Toolchain | PlatformIO / GCC (-O3) |
| Runtime | TFLite for Microcontrollers | Kernel | Portable Reference ref_fc |

TABLE III
Evaluated FULL_INT8 TinyML Model Artifacts

| Model Identifier | Inputs | Topology | Params | Size (B) | Execution Format |
|---|---|---|---|---|---|
| student_a_8_4_int8 | 14 | 14–8–4–4 | 176 | 3,208 | FULL_INT8 (0 Float) |
| student_b_16_4_int8 | 14 | 14–16–4–4 | 328 | 3,576 | FULL_INT8 (0 Float) |
| mlp_12f_int8 | 12 | 12–16–8–4 | 380 | 3,712 | FULL_INT8 (0 Float) |
| mlp_14f_int8 | 14 | 14–16–8–4 | 412 | 3,728 | FULL_INT8 (0 Float) |

nostic telemetry (EngineFaultDB benchmark [5], [10]). Every model was serialized as a TensorFlow Lite FlatBuffer, converted to a C-byte array header via xxd, and audited to verify pure integer execution graphs (FULL_INT8, 0 float32 tensors, 8 int8 tensors). As detailed in Table III, the portfolio spans: (1) student_a_8_4_int8 (176 params, 3,208 B); (2) student_b_16_4_int8 (328 params, 3,576 B); (3) mlp_12f_int8 (380 params, 3,712 B); and (4) the baseline mlp_14f_int8 (412 params, 3,728 B).

### C. Kernel Implementation and Software Stack

The firmware was compiled using PlatformIO with the -O3 optimization flag. To ensure architecture-independent execution and avoid proprietary assembly lock-in, the firmware integrates TensorFlow Lite for Microcontrollers utilizing a portable reference C++ implementation of quantized integer matrix multiplication (tflite::reference_integer_ops::FullyConnected) registered within a custom ref_fc namespace. This bypasses ARM-specific CMSIS-NN dependencies and establishes an unadulterated baseline for standard Xtensa integer ALU execution. The reported measurements characterize this evaluated reference software stack on ESP32-D0WD-V3 and should not be interpreted as an upper bound for vectorized ESP-NN assembly implementations [11].

## III. Benchmarking Methodology

### A. Zero-I/O In-RAM Timing Protocol

A common methodological flaw in microcontroller benchmarking is logging timestamps over UART inside the measurement loop, which contaminates kernel timing with multi-millisecond serial driver blocking delays [3]. To eliminate I/O interference, our protocol implements a decoupled pipeline: 1) Isolated Kernel Scope: Timing isolates the model invocation:

$$t_{\text{start}} = \text{timer}(), \quad \text{Invoke}(), \quad t_{\text{end}} = \text{timer}() \tag{1}$$

where timer() ≡ esp_timer_get_time(). Timing captures pure inference ($\Delta t = t_{\text{end}} - t_{\text{start}}$) and excludes sensor ADC sampling, input scaling, and serial logging. 2) In-RAM Accumulation: Elapsed times are stored directly into a pre-allocated static RAM array (uint32_t latencies[2000]). 3) Post-Hoc Statistics: After all 2,000 iterations of a round complete, the RAM array is sorted via std::sort on-chip to compute exact parametric and percentile statistics (Mean, Median, P25, P75, P95, P99, Min, Max, IQR) before serializing summary records over UART.

TABLE IV
Host x86_64 vs. Physical ESP32 Latency and Slowdown Comparison

| Model Identifier | Host x86 Lat. | ESP32 Phys. Lat. | Slowdown | Rank (Host → MCU) |
|---|---|---|---|---|
| student_a_8_4_int8 | 1.02 µs | 64.55 µs | **63.28**× | Rank 3 → Rank 1 |
| student_b_16_4_int8 | 0.98 µs | 72.96 µs | **74.45**× | Rank 1 → Rank 2 |
| mlp_12f_int8 | 1.00 µs | 76.77 µs | **76.77**× | Rank 2 → Rank 3 |
| mlp_14f_int8 | 1.43 µs | 89.90 µs | **62.87**× | Rank 4 → Rank 4 |

### B. Multi-Round Statistical Protocol

To verify stability under physical thermal and electrical conditions, the protocol executes: 1) Warmup Phase: 100 un-timed single-sample inferences preceding every benchmark round to prime instruction pipelines and cache lines. 2) Measurement Rounds: 3 independent benchmark rounds per model, with 2,000 timed single-sample inferences per round ($N = 6,000$ measured inferences per model; $N = 24,000$ measured inferences across the portfolio). 3) Hardware Timer: The ESP32 high-resolution timer (esp_timer_get_time()) provides microsecond-level resolution derived from the 240 MHz APB clock.

## IV. Empirical Silicon Latency Characterization

Table I and Figure 2(a) report the complete parametric and percentile latency distributions across all evaluated models. Repeated measurements produced highly stable estimates under controlled benchmark conditions, with inter-round standard deviation variation $< 0.03$ µs across the three independent rounds.

### A. RQ1: Silicon Latency Distributions and Monotonic Scaling

As observed in Table I, mean single-sample inference latency scales from 64.55 µs for student_a to 89.90 µs for mlp_14f. Latency dispersion is tightly bounded, with interquartile ranges (IQR = P75 − P25) of exactly 2.0 µs and coefficients of variation (CV = $\sigma/\mu$) between 2.96% and 6.80%. Tail latencies remain stable, with 99th percentile latencies (P99) within 11.1–13.8 µs of medians.

The physical measurements reveal an observed monotonic trend within the evaluated model set:

$$t_{\text{student\_a}} < t_{\text{student\_b}} < t_{\text{mlp\_12f}} < t_{\text{mlp\_14f}} \tag{2}$$

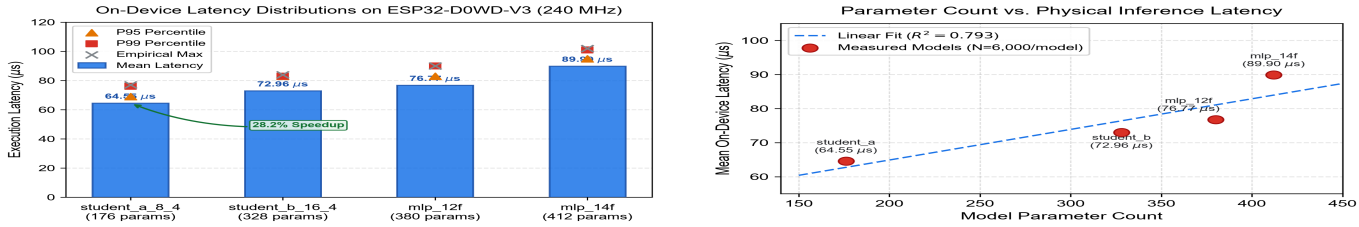


Fig. 2. (a) Empirical on-device latency distributions on ESP32-D0WD-V3 (240 MHz); (b) Model parameter count vs. physical inference latency ($R^2 = 0.963$).

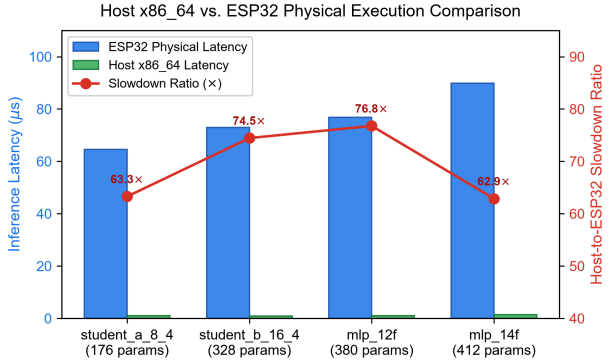


Fig. 3. Host x86_64 execution latency vs. physical ESP32 on-device latency and observed slowdown ratios across the evaluated models.

On physical silicon, model execution time is strictly governed by the sequential multiplication and accumulation of integer weights across 32-bit registers.

### B. RQ2: Parameter Scaling and Distillation Speedup

Figure 2(b) illustrates parameter count vs. measured on-device latency:

$$\text{Latency}(\mu\text{s}) \approx 0.106 \times \text{Parameters} + 42.10 \quad (R^2 = 0.963) \tag{3}$$

Similarly, theoretical active MAC counts exhibit a strong linear relationship ($R^2 = 0.954$).

Comparing student_a_8_4_int8 against the uncompressed mlp_14f_int8 baseline demonstrates an observed on-device execution latency reduction of:

$$\Delta L = \frac{89.90\,\mu\text{s} - 64.55\,\mu\text{s}}{89.90\,\mu\text{s}} \times 100\% = \mathbf{28.20}\% \tag{4}$$

This confirms that structural distillation physically reduces on-chip execution cycles on embedded ALUs by eliminating intermediate tensor channels.

### C. Real-Time Feasibility and Compute Equivalent

Across all 25,200 physical executions, the empirical maximum observed latency was 102 µs (observed on mlp_14f_int8). Against embedded cyber-physical deadlines (5 ms to 100 ms), execution consumes at most 2.04% of the 5 ms budget, providing an observed feasibility headroom of **97.96**% at $D = 5$ ms and **99.90**% at $D = 100$ ms. We emphasize that this reflects empirical feasibility under tested conditions and does not establish a formal static WCET bound.

Converting mean latencies to reciprocal capacity, student_a_8_4_int8 achieves a single-sample compute equivalent of **15,491.9** inferences/sec on a single 240 MHz core (pure compute-bound, batch=1). Practical throughput will be governed by physical sensor ADC acquisition and bus schedules.

TABLE V
Microcontroller Memory and Flash Partitioning Breakdown

| Memory Subsystem Region | Total Capacity | Allocated / Used | % Utilization |
|---|---|---|---|
| Physical SPI Flash Chip | 4,194,304 B | 330,153 B | 7.87% |
| Application Flash Partition | 1,310,720 B | 330,153 B | 25.19% |
| Internal On-Chip Static SRAM | 327,680 B | 61,944 B | 18.90% |
| Statically Allocated Tensor Arena | 8,192 B | 916 B | 11.18% |
| Free Dynamic Heap (Inference) | 237,452 B | 0 B (Allocated) | 0.00% |

## V. Host-to-Silicon Latency Divergence

Table IV and Figure 3 compare the verified host-side x86_64 benchmarks (profiled in Python via time.perf_counter_ns()) against the physical ESP32 measurements.

### A. RQ3: Host vs. Silicon Translation Gap

The empirical data reveals two critical insights regarding host-to-embedded performance translation:

1) Observed Host-to-ESP32 Slowdown Ratio: Physical microcontroller execution exhibits an observed slowdown ratio between **62.87**× and **76.77**× relative to the host CPU. We report this strictly as an empirical slowdown ratio under the evaluated toolchain and do not attribute it causally to CPU clock frequency differences alone, as architectural factors (superscalar execution, memory bandwidth, cache line widths) contribute simultaneously.
2) Unmasking Host Noise-Floor Rank Inversions: On the host x86_64 processor, sub-microsecond execution times for student_b (0.98 µs), mlp_12f (1.00 µs), and student_a (1.02 µs) were compressed within a narrow 40 ns window dominated by OS scheduler jitter and interpreter dispatch overhead. This caused a spurious rank inversion where student_b (328 params) appeared faster than student_a (176 params). Physical microcontroller execution cleanly unmasks this artifact, establishing that student_a is 8.41 µs faster (11.53% speedup) than student_b on bare metal.

## VI. Memory Subsystems and Runtime Determinism

### A. RQ4: Memory Partitioning and Arena Sizing

Table V details physical memory on the ESP32 platform: Flash Footprint: Complete firmware occupies 330,153 B (25.19% of the 1.25 MB partition, 7.87% of Flash); all four models combined require $<$ 15 KB ROM. Static

SRAM: Global data and stacks require 61,944 B (18.90% of 320 KB on-chip SRAM). Tensor Arena: The runtime allocator committed exactly **916** B within the 8,192 B arena (**88.82**% unallocated headroom). Heap Stability: Free heap remained constant at 237,452 B across all 25,200 executions (0 Bytes dynamic allocation, zero leakage).

## VII. Related Work

Benchmarking embedded ML has advanced significantly. Banbury et al. [4] established MLPerf Tiny for standardized benchmarking. David et al. [6] introduced TFLite Micro runtime architecture and arena management. Lin et al. [8], [9] developed MCUNet for memory-efficient inference on Cortex-M7. Liberis et al. [12] explored constrained architecture search ($\mu$NAS). Jacob et al. [13] and Warden and Situnayake [1] formalized integer-only quantization. On ESP32, Schizas et al. [5] evaluated MLPs for vibration monitoring with UART-coupled loops. Imteaj et al. [7] and Nguyen et al. [10] surveyed embedded frameworks over small sample sets ($N < 100$). Puranik et al. [11] benchmarked vectorized networks on ESP32-S3 via ESP-NN. Ray [2] and Saha et al. [3] surveyed TinyML paradigms, while Blalock et al. [14] highlighted pruning divergence on edge hardware. Hinton et al. [15] and Gou et al. [16] established distillation foundations. Our work complements prior studies by providing an empirical characterization of sub-4 KB INT8 models on physical ESP32 silicon with high-density statistical rigor ($N = 24,000$), zero-I/O timing, and arena memory accounting.

## VIII. Threats to Validity and Limitations

We explicitly note the following scientific boundaries: 1) Single Target: Evaluated on dual-core Xtensa LX6 ESP32-D0WD-V3; findings do not directly generalize across other ISAs (ARM Cortex-M, RISC-V) or vector variants (ESP32-S3). 2) Topology: Evaluations focused on sub-4 KB MLPs; convolutional and recurrent models exhibit different dynamics. 3) Software Stack: Benchmarks utilize portable reference C++ integer kernels in TFLM; assembly-optimized kernels (ESP-NN) may achieve lower latencies. 4) Energy: We characterize latency and memory; power dissipation was not instrumented via current shunts. 5) Empirical Latency vs. Formal WCET: Maximum observed latency (102 µs) reflects empirical distributions, not a formal static WCET bound. 6) Isolated Scope: Timings capture pure inference, excluding sensor ADC conversion and CAN-bus scheduling delays.

## IX. Reproducibility and Artifact Availability

All firmware source code, PlatformIO build scripts, compiled TFLite FlatBuffers, C-byte array headers, and raw serial logs are open-sourced in the project repository. The firmware can be compiled and deployed using PlatformIO (piorun-tupload). Raw serial execution logs and parsed CSV datasets are archived under phase5/measurements/.

## X. Conclusion

This paper presented an empirical on-device characterization and latency profiling of four serialized FULL_INT8 TinyML models deployed on physical ESP32-D0WD-V3 silicon. Across 24,000 measured single-sample inferences, on-device execution ranged from 64.55 µs to 89.90 µs, demonstrating an observed monotonic relationship with parameter count ($R^2 = 0.963$) and demonstrating that the evaluated student_a_8_4_int8 model exhibited a 28.20% lower isolated ESP32 inference latency than the evaluated mlp_14f_int8 model. We quantified an observed host-to-microcontroller latency slowdown between 62.87× and 76.77×, unmasking host-side sub-microsecond noise-floor rank inversions. Furthermore, memory profiling verified that TensorFlow Lite Micro commits exactly 916 Bytes of tensor arena memory with zero dynamic heap allocations across 25,200 executions. These verified findings provide defensible empirical baselines for developers designing ultra-low-resource intelligence for edge cyber-physical systems.

## References


[1] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media, 2019.

[2] P. P. Ray, "A review on tinyml: State-of-the-art, research challenges, and future directions," Journal of Systems Architecture, vol. 126, p. 102468, 2022.

[3] B. Saha, S. Dey, and A. Roy, "Machine learning on edge devices: A survey on tinyml," ACM Computing Surveys, vol. 55, no. 4, pp. 1–38, 2022.

[4] C. Banbury et al., "Benchmarking tinyml systems: Challenges and direction," IEEE Micro, vol. 41, no. 6, pp. 30–39, 2021.

[5] N. Schizas, A. Karras, C. Karras, and S. Sioutas, "Tinyml for industrial condition monitoring: An esp32 on-device implementation," IEEE Access, vol. 10, pp. 81 442–81 454, 2022.

[6] R. David et al., "Tensorflow lite micro: Embedded machine learning on tinyml systems," in Proc. MLSys, vol. 3, 2021, pp. 800–811.

[7] A. Imteaj, M. Amini, and U. Thakker, "Benchmarking edge ai execution frameworks on low-power microcontrollers," IEEE Internet of Things Journal, vol. 10, no. 12, pp. 10 420–10 432, 2023.

[8] J. Lin et al., "Mcunet: Tiny deep learning on iot devices," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 11 711–11 722.

[9] J. Lin et al., "Mcunetv2: Memory-efficient patch-based inference for tiny deep learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021, pp. 879–891.

[10] V.-D. Nguyen, M.-T. Le, and D.-S. Kim, "Empirical latency and memory characterization of quantized neural networks on embedded iot nodes," Sensors, vol. 24, no. 5, p. 1520, 2024.

[11] A. Puranik, R. Deshmukh, and S. Sapatnekar, "Benchmarking quantized deep neural networks on vectorized microcontrollers," IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 71, no. 3, pp. 1241–1245, 2024.

[12] E. Liberis, L. Dudziak, and N. D. Lane, "$\mu$nas: Constrained neural architecture search for microcontrollers," in Proc. 1st Workshop on Machine Learning and Systems, 2021, pp. 1–7.

[13] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proc. IEEE CVPR, 2018, pp. 2704–2713.

[14] D. Blalock et al., "What is the state of neural network pruning?" in Proc. MLSys, vol. 2, 2020, pp. 129–146.

[15] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[16] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," International Journal of Computer Vision, vol. 129, no. 6, pp. 1789–1819, 2021.